Treasury Management DAO Report

Prasanna Sai Puvvada, pp2689
Kohsheen Tiku, kt2761

Table of Contents:

Objective 3
Description 3
Rationale Behind Selecting This Project 3
Expectations 3
Code Documentation 4
I. MemberToken Contract - MemberToken.sol 4
II. Gold Token, Land Token, and Bank Token Contracts (GoldToken.sol, LandToken.sol, and BankToken.sol) 4
III. TreasuryManagement.sol 4
IV. MembersDAO.sol 5
Execution Flow 6
Screenshots & Links: 8
A. Gold Token 8
B. Land Token 9
C. Bank Token 9
D. Treasury Management 10
E. Members Dao 10
F. Members Token 11
Execution of Contract (Video) 12
Learnings 12
Conclusion 12
References 13

Objective

The objective of the chosen project is to create a Decentralized Autonomous Organization (DAO) for investment management funds, where voting determines the allocation of funds into various investment opportunities. The project aims to demonstrate the power of decentralized decision-making in investment management, providing transparency, fairness, and inclusivity to all participants. Through this DAO structure, participants can propose investment opportunities, vote on them, and execute investments based on the collective decision of the DAO members.

Description

The project entails the implementation of smart contracts on a blockchain platform to establish the DAO infrastructure and manage token transactions. Members of the DAO will be able to propose investment decisions, such as acquiring Land, Gold, or Bank tokens, and vote on these proposals. The voting process will be governed by the participants who hold the Member token, each participant holds equal rights. Once a proposal is approved through voting, the corresponding Land, Gold, or Bank tokens will be transferred to the member's DAO contract which will be given by the treasury management contract in exchange of the Sepolia ETH.

Rationale Behind Selecting This Project

1. Innovative Asset Management: The project introduces a novel approach to asset management within a decentralized ecosystem by incorporating various tokenized assets (Land, Gold, Bank) and facilitating their distribution based on community decisions through voting.
2. Decentralized Governance: By leveraging DAO principles, the project empowers community members to actively participate in investment decisions and governance processes, promoting transparency, fairness, and inclusivity.
3. Token Economy: The project's use of multiple tokens (Land, Gold, Bank, and Member) demonstrates the versatility of blockchain-based token economies in managing diverse assets and incentivizing participation within the DAO ecosystem.
4. Financial Inclusion: Through the decentralized investment management DAO, the project aims to lower barriers to entry and enable broader participation in investment opportunities, thereby promoting financial inclusion and empowerment.

Expectations

1. Successful deployment of smart contracts for the DAO infrastructure, including token creation, voting mechanisms, and asset management functionalities.
2. Active participation from members in proposing investment decisions, casting votes, and engaging in community discussions within the DAO ecosystem.
3. Efficient execution of approved proposals, with Land, Gold, or Bank tokens being transferred to members' DAO contracts in exchange for cryptocurrency from the treasury management contract.

4. Transparent and auditable governance processes, ensuring that decisions are made democratically and in the best interest of the DAO community.

Code Documentation

The project comprises **two main smart contracts**: ‘TreasuryManagement’ and ‘MembersDAO’ and **4 token contracts**: ‘LandToken’, ‘BankToken’, ‘GoldToken’ and ‘MemberToken’. These contracts are deployed on the Ethereum blockchain and interact with each other to execute the DAO's investment decisions.

I. MemberToken Contract - MemberToken.sol

The MemberToken contract represents a unique ERC20 token that serves as a governance token within a DAO i.e it acts as a voting right for a member, having this token ensures a member is allowed to vote. The contract inherits from OpenZeppelin's ERC20 for standard token functionality, Ownable for ownership management, and ERC20Permit for gasless transactions.

- Constructor: Initializes the token with a name ("MemberToken") and symbol ("MBT"). The Ownable constructor is called with the initialOwner address, transferring ownership of the contract to the specified address. The ERC20Permit is initialized with the token name to enable permit functionality.
- Mint Function: Allows the contract owner to mint new tokens to a specified address. The amount is adjusted by the token's decimal count to maintain token denomination integrity.

II. Gold Token, Land Token, and Bank Token Contracts (GoldToken.sol, LandToken.sol, and BankToken.sol)

These contracts are specific to the assets they represent: land, gold, and bank assets, respectively. Each contract allows the owner to mint tokens, providing a mechanism to increase the supply of tokens in circulation to designated addresses.

- Constructor: Similar to MemberToken, each constructor initializes with a unique name and symbol for the respective token, sets the initialOwner through the Ownable constructor, and prepares the token for permit-based operations with ERC20Permit.
- Mint Function: Identical across all contracts, enabling the contract owner to issue new tokens.

III. TreasuryManagement.sol

This contract manages the treasury of the DAO. It allows for the purchase and allocation of asset tokens (Gold, Land, Bank) based on the decisions made by DAO members.

Key Functions:
- Constructor: Initializes the contract by setting the owner and the contract's address as the treasury management address.

- transferToken(uint decision, uint amount, address token, address members): Facilitates the purchase of tokens. It takes the investment decision, the amount of tokens to be bought, the token contract address, and the recipient's address as parameters. Depending on the decision, it interacts with the respective token contract (GoldToken, LandToken, BankToken) to mint and transfer tokens.

Parameters: The function accepts several parameters to define the investment action:

- decision: Numerical representation of the investment choice (0 for Gold, 1 for Land, 2 for Bank).
- amount: The quantity of tokens to be purchased and allocated.
- token: Contract address of the specific asset token.
- members: Recipient address for the allocated tokens (MembersDao address or wallets or contracts).

Execution Flow:

- Validation:
    - Confirms the transaction includes sufficient Ether for the purchase, based on a predefined rate (0.001 ETH per token as a placeholder).
- Asset Allocation:
    - Depending on the decision, it dynamically interacts with the corresponding token contract (GoldToken, LandToken, BankToken).
    - Checks if the treasuryManagement contract's balance of the chosen token is less than the desired amount. If so, it mints tokens to meet the requirement.
    - Approves and transfers the specified amount of tokens from the contract to the members address.

Interaction with TreasuryManagement

The TreasuryManagement contract is pivotal in executing the DAO's investment decisions. It handles the purchasing of tokens (gold, land, bank) based on the DAO's vote, minting these tokens, and transferring them accordingly. The MembersDAO contract sends ether and specifies the investment choice through the transferToken function call to TreasuryManagement.

IV. MembersDAO.sol

This contract represents the governance layer of the DAO, allowing members to vote on investment proposals and execute the collective decision.

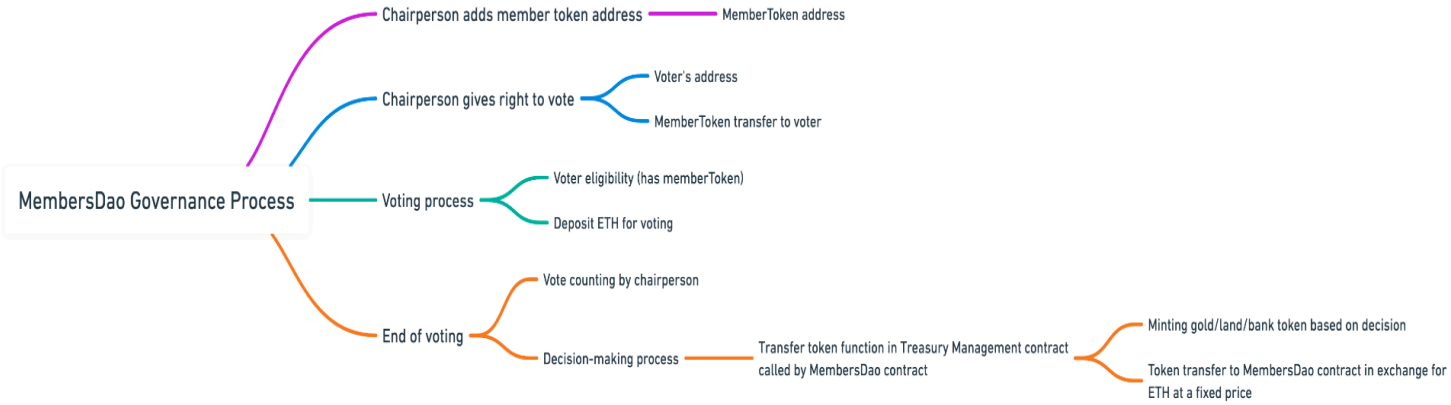
Key Functions:

- Constructor: Sets up the DAO, including the treasury management address, token addresses (gold, land, bank), and initializes voting proposals.
- DepositEth(): Allows members to deposit ether into the contract for future investments.
- membersTokenId(address memberAddress): Associates member tokens with DAO members, granting them voting rights.
- giveRightToVote(address voter): Grants voting rights to a member by transferring them a MemberToken (single token), enabling them to participate in the voting process.
- vote(uint proposal): Records a vote for one of the investment proposals and takes back the granted MemberToken.
- countVote(): Tallys the votes to determine the winning proposal after the voting period has ended.
- EndVote(): Executes the investment decision by interacting with the 'TreasuryManagement' contract to mint and allocate the chosen asset tokens.

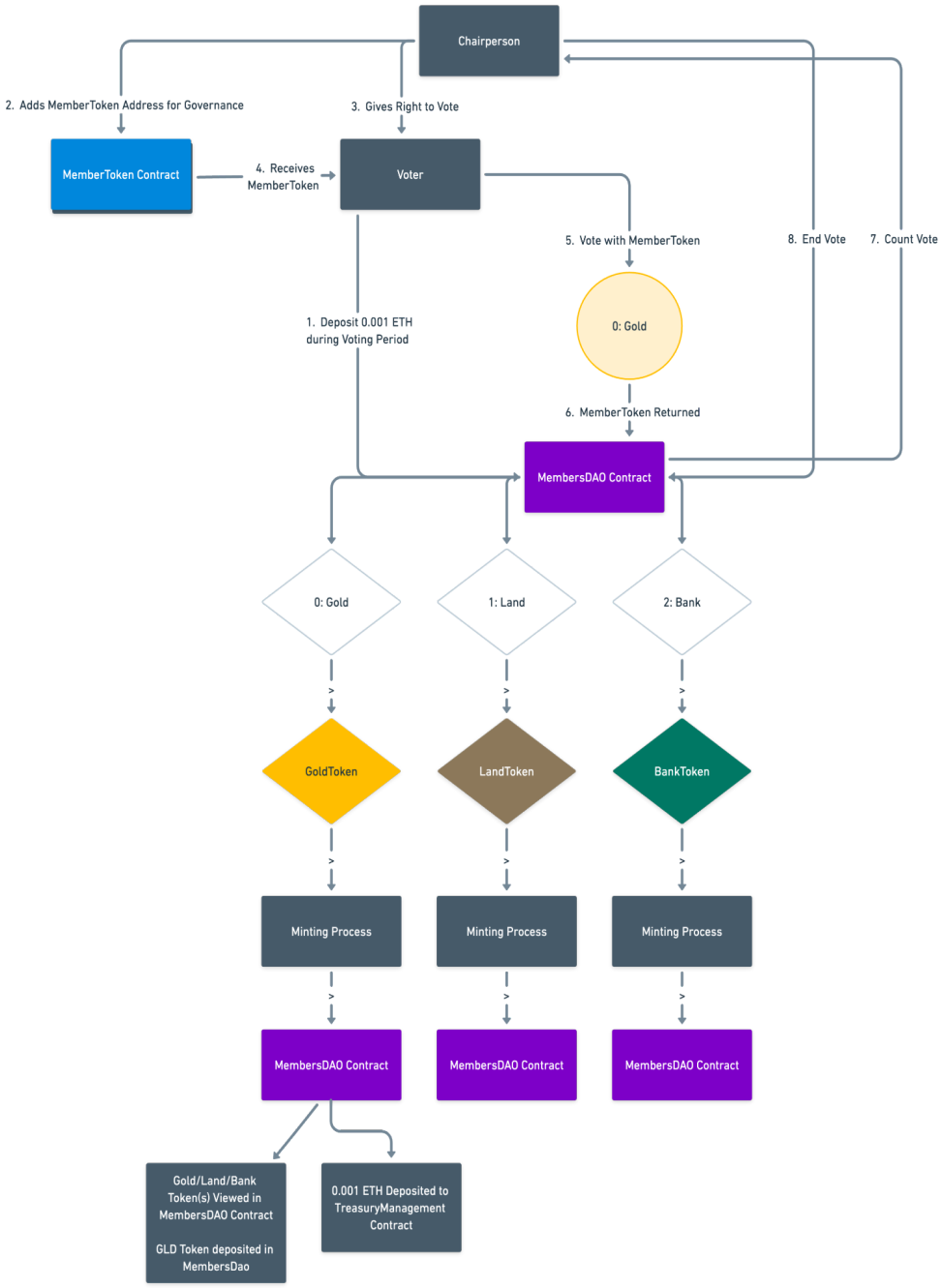
Error Handling

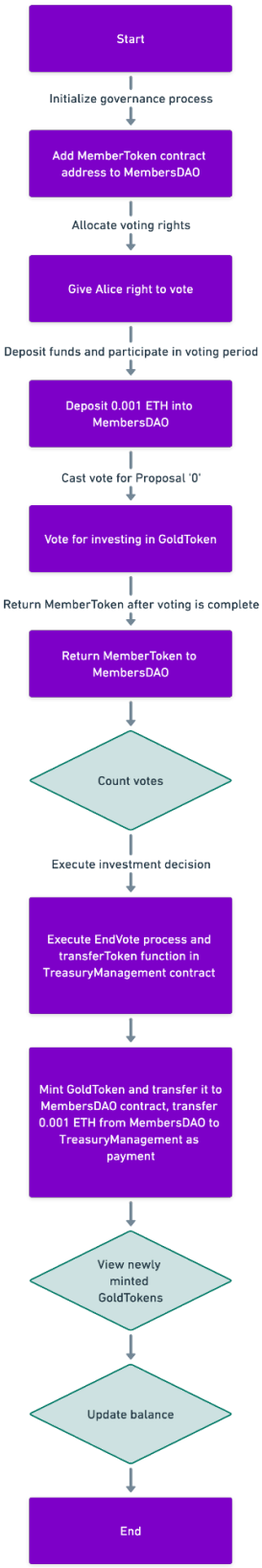
The contract employs custom errors (voteAlreadyEnded, voteNotYetEnded) for managing exceptions related to the voting timeline, ensuring that operations are executed within the appropriate period.

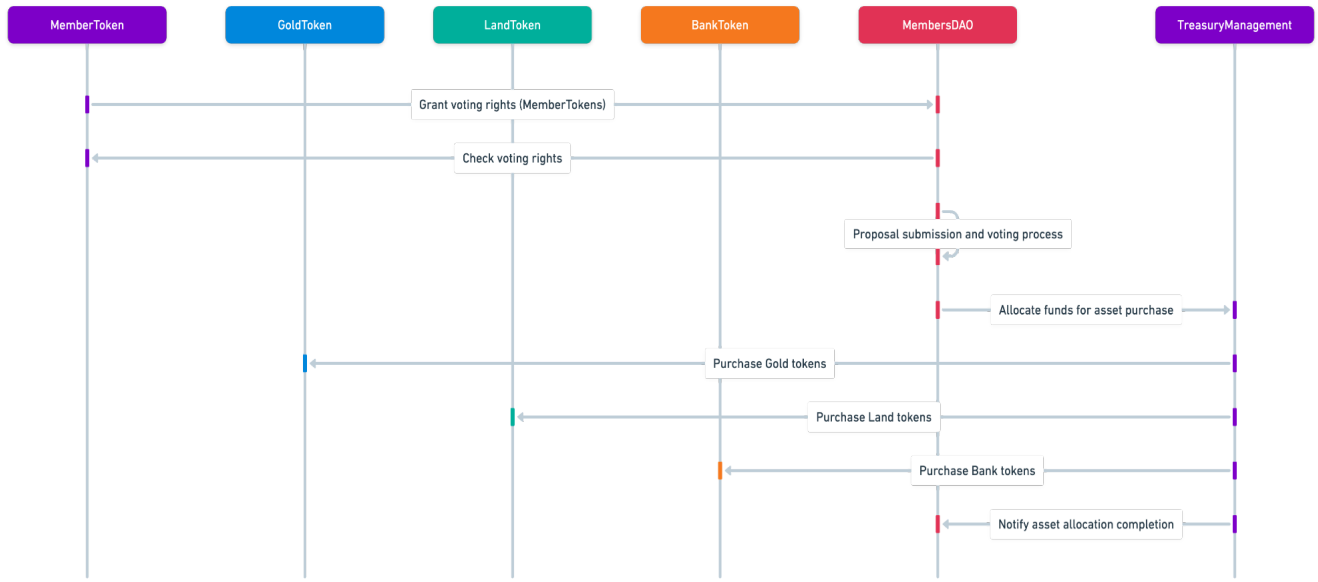
Execution Flow



Architecture:







1. Chairperson of MembersDao adds member token address to consider which is decided for governance.
2. Chairperson then gives right to vote to a voter using the voter's address and the voter receives a memberToken from the MemberDao contract.
3. A voter can vote only if he/she has a memberToken.
4. Once a voter has voted the memberToken is taken back and is transferred to MembersDao contract.
5. Members can deposit funds in MembersDao contract using deposit ETH function only during the voting time period.
6. Once the voting time is completed, the chairperson will count the vote.
7. End vote will take place after count vote which will be done by chairperson
8. If decision is taken the MembersDao contract will internally call transfertoken function in treasury management contract
9. The Treasury management contract will see what is the decision and based on the decision it will mint gold/land/bank token and then transfer it to MembersDao contract in exchange of ETH at a price of 0.001 ETH per gold/land/bank token.
10. At the moment the code requests for only 1 token at a time after the endnote and does not give an option to choose how many tokens are required.
11. Once the transaction is complete, gold/land/bank token(s) can be viewed in the MembersDao contract.

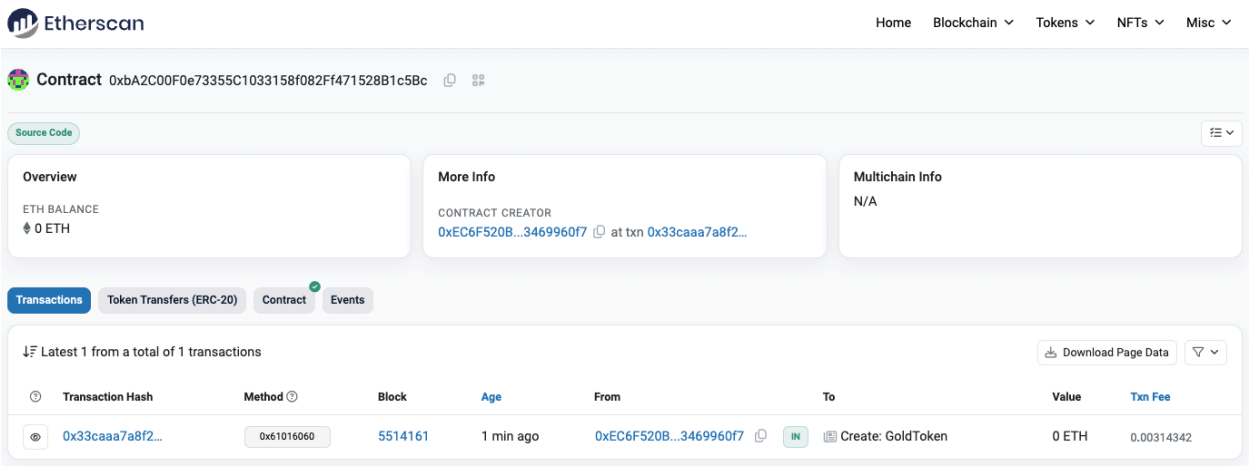
Screenshots & Links:

A. Gold Token

Etherscan link:

https://sepolia.etherscan.io/address/0xba2c00f0e73355c1033158f082ff471528b1c5bc

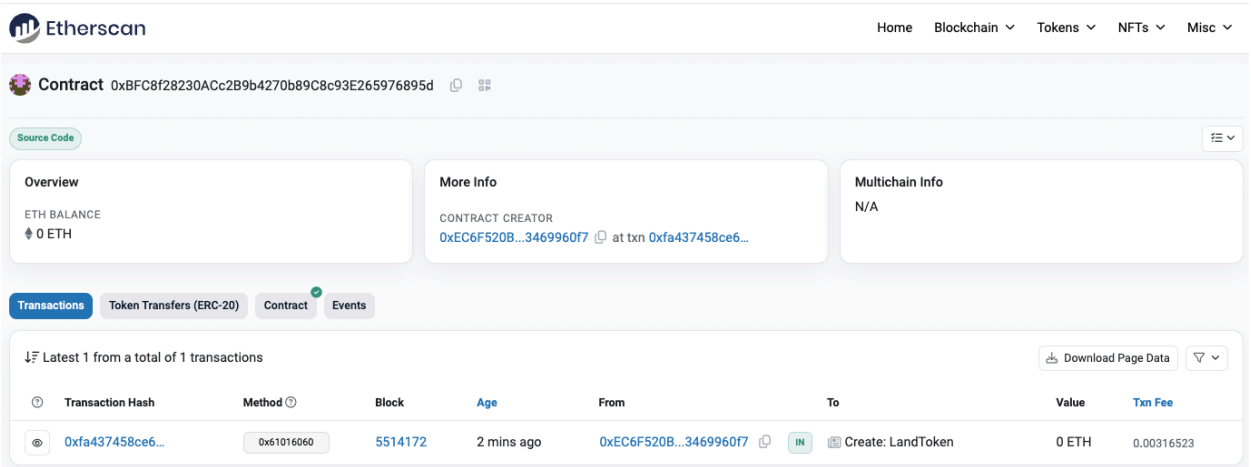
Contract verified and published:



B. Land Token

Etherscan link:
https://sepolia.etherscan.io/address/0xbfc8f28230acc2b9b4270b89c8c93e265976895d

Contract verified and published:

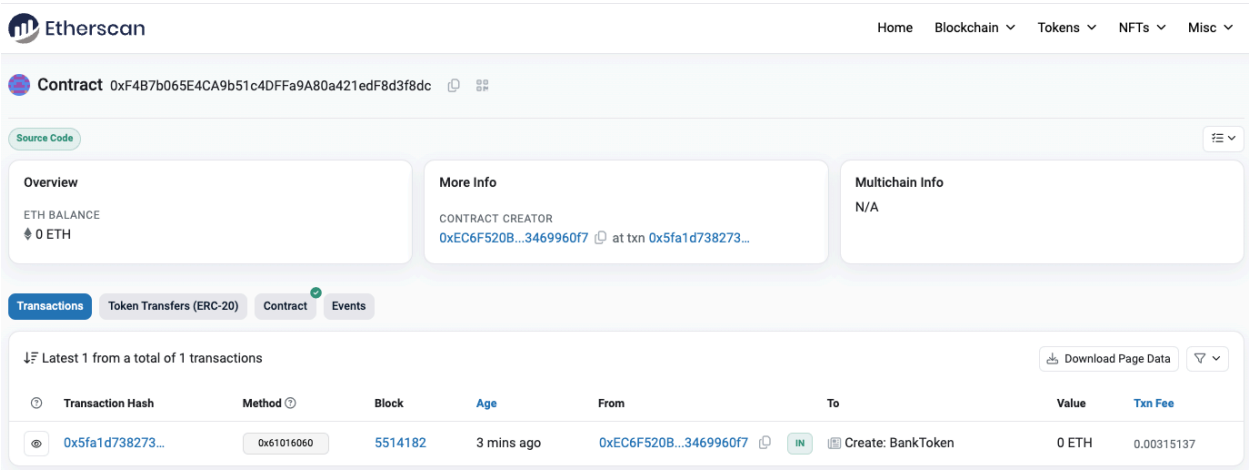


C. Bank Token

Etherscan link:

https://sepolia.etherscan.io/address/0xf4b7b065e4ca9b51c4dffa9a80a421edf8d3f8dc

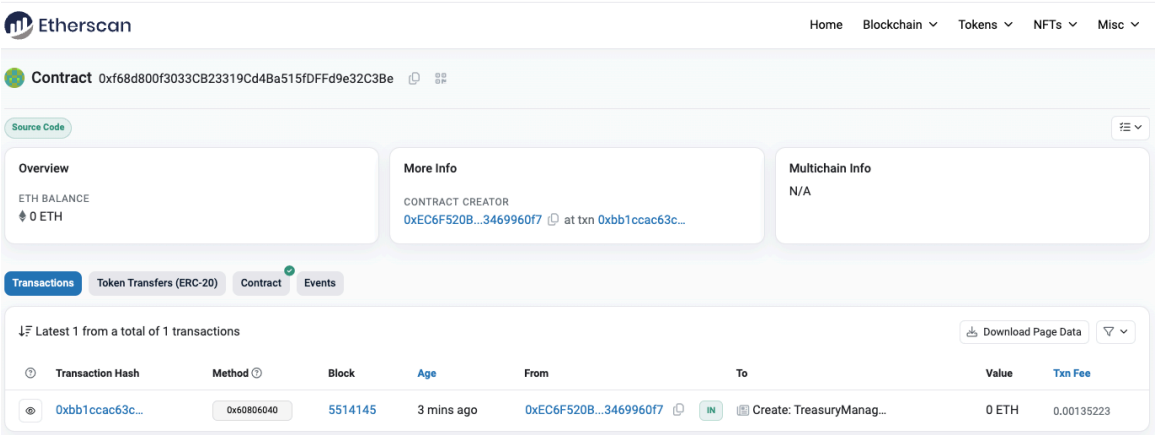
Contract Verified and published:



D. Treasury Management

Etherscan link:
https://sepolia.etherscan.io/address/0xf68d800f3033cb23319cd4ba515fdffd9e32c3be

Contract Verified and published:



E. Members Dao

Etherscan Link:
https://sepolia.etherscan.io/address/0x001daa12b0b2e5cc5992c8778714286b4abd9c32

Contract Verified and published:

Etherscan

Home Blockchain Tokens NFTs Misc

Contract 0x001DAa12B0B2E5CC5992c8778714286b4aBD9c32

Source Code

Overview

ETH BALANCE
0 ETH

TOKEN HOLDINGS
$0.00 (2 Tokens)

More Info

CONTRACT CREATOR
0xEC6F520B...3469960f7 at txn 0xad43e43fa8d...

Multichain Info
N/A

Transactions | Internal Transactions | Token Transfers (ERC-20) | Contract | Events

Latest 7 from a total of 7 transactions

Download Page Data

Transaction Hash	Method	Block	Age	From		To	Value	Txn Fee
0xb0265d191c...	End Vote	5514311	7 mins ago	0xEC6F520B...3469960f7	IN	0x001DAa12...b4aBD9c32	0 ETH	0.00023521
0xa7bb8dda41...	Count Vote	5514302	9 mins ago	0xEC6F520B...3469960f7	IN	0x001DAa12...b4aBD9c32	0 ETH	0.00006459
0x7fd9a574f37...	Vote	5514294	10 mins ago	0x0559Fd52...DCDcC7a5a	IN	0x001DAa12...b4aBD9c32	0 ETH	0.00017073
0x7f722e43ce5...	Give Right To ...	5514289	11 mins ago	0xEC6F520B...3469960f7	IN	0x001DAa12...b4aBD9c32	0 ETH	0.00021391
0x8ac724c7da...	Members Tok...	5514283	13 mins ago	0xEC6F520B...3469960f7	IN	0x001DAa12...b4aBD9c32	0 ETH	0.00006881
0x38d8e0b26fe...	Deposit Eth	5514277	14 mins ago	0xEC6F520B...3469960f7	IN	0x001DAa12...b4aBD9c32	0.001 ETH	0.00007149
0xad43e43fa8d...	0x60806040	5514266	16 mins ago	0xEC6F520B...3469960f7	IN	Contract Creation	0 ETH	0.00331001

F. Members Token

Etherscan Link:
https://sepolia.etherscan.io/address/0xb9b2cd0a05da8c328d930472b53482417583cc1e
Contract Verified and published:

Etherscan

Home Blockchain Tokens NFTs Misc

Contract 0xb9B2cD0a05dA8c328d930472B53482417583cC1E

Source Code

Overview

ETH BALANCE
0 ETH

More Info

CONTRACT CREATOR
0xEC6F520B...3469960f7 at txn 0xcdba05cabcf...

TOKEN TRACKER
MemberToken (MBT)

Multichain Info
N/A

Transactions | Token Transfers (ERC-20) | Contract | Events

Latest 1 from a total of 1 transactions

Download Page Data

Transaction Hash	Method	Block	Age	From		To	Value	Txn Fee
0xcdba05cabcf...	0x61016060	5514273	14 mins ago	0xEC6F520B...3469960f7	IN	Contract Creation	0 ETH	0.00313101

Execution of Contract (Video)

Check the video:
https://drive.google.com/file/d/1BzGFu5yMD_t3FomKQCf9M6XReeJWcMMR/view?usp=drive_link

Learnings

Understanding the DAO Framework: Began with a thorough examination of the DAO's purpose, focusing on its goal to democratize investment management through collective decision-making.

Delved into the Solidity code to understand the mechanics behind token distribution, voting rights allocation, the voting process, and the execution of investment decisions. This step was crucial for comprehending how the DAO operates from initiation to conclusion of investment decisions.

Formulating the procedural workflow of the DAO, detailing each phase from the chairperson initializing the governance process, through member participation in voting, to the eventual execution of investment decisions based on the collective will of the DAO members.

Conclusion

In conclusion, the exploration of the Decentralized Autonomous Organization (DAO) for investment management has illuminated the intricate mechanisms and significant potential of blockchain technology in facilitating democratic, transparent, and efficient governance and investment strategies. Through the detailed examination and documentation of the DAO's smart contracts and operational workflow, several key insights and learnings have emerged, underscoring the transformative impact of decentralized governance on investment management. The process highlighted the importance of clear, secure, and efficient smart contract interactions, which form the backbone of DAO operations, enabling collective decision-making and the execution of investment strategies.

To accomplish the project on creating a Decentralized Autonomous Organization (DAO) for investment management, the process commenced with establishing the foundational smart contracts: MembersDAO for governance, TreasuryManagement for financial executions, and individual contracts for asset tokens (GoldToken, LandToken, BankToken). The governance framework was set by integrating the MemberToken contract to grant voting rights. Following this, the workflow involved distributing MemberTokens to enable members to vote on investment proposals. Deposits in ETH were collected during the voting period as investment capital. Upon conclusion of voting, the chairperson tallied votes to identify the winning investment proposal. The execution phase was initiated by the MembersDAO contract, instructing the TreasuryManagement contract to mint and allocate the chosen asset tokens in exchange for the deposited ETH, culminating in the successful execution of the DAO’s collective investment decision. This streamlined process not only illustrated the efficacy of blockchain in democratizing investment decisions but also showcased the potential for transparent and inclusive financial management within a decentralized framework.

Overall, this exploration into the DAO's framework for investment management has not only expanded understanding of decentralized finance (DeFi) and blockchain applications but also sparked curiosity about the future possibilities and improvements in DAO governance and operations. The lessons learned from this assignment underscore the evolving nature of investment management in the digital age and the growing importance of community-driven decision-making processes.

References

- https://github.com/partylikeits1983/simpleDAO/tree/main
- https://app.bunzz.dev/module-templates/afa05522-5f39-4850-baf3-3390fd139f54?version=latest
- https://docs.openzeppelin.com/contracts/5.x/wizard